# Modellazione delle prestazioni di un DBMS a partire da una traccia di workload

Marco Chiodo, Giuseppe Casteltrione

Relazione di progetto 3 – Tecnologie e Architetture per la Gestione dei Dati

## 1 Introduzione

La progettazione e il dimensionamento dell'infrastruttura hardware di un DBMS richiedono l'utilizzo di strumenti quantitativi in grado di stimare, già in fase preliminare, l'impatto che eventuali variazioni del carico di lavoro o della configurazione del sistema possono avere sulle prestazioni osservate dagli utenti.

La teoria delle code rappresenta un approccio consolidato per l'analisi di tali problematiche, poiché consente di modellare il comportamento del sistema a partire da parametri misurabili, come l'utilizzo delle risorse, i tempi di servizio e le caratteristiche del traffico generato. Attraverso la costruzione di un modello analitico è quindi possibile: valutare le prestazioni del sistema, confrontare le previsioni con i dati rilevati e supportare le decisioni di capacity planning, riducendo la necessità di effettuare modifiche dirette sull'ambiente di produzione con rischi e costi elevati.

In questo lavoro si applica tale approccio alla modellazione delle prestazioni di un DBMS, a partire da una traccia di workload reale (`transactions.csv`) raccolta osservando il sistema per un periodo di 24 ore, durante il quale sono state registrate $N = 100\,000$ transazioni. Per ciascuna transazione la traccia riporta il tempo di CPU richiesto, il numero di accessi a un array di dischi in configurazione RAID-0 e la percentuale di tali accessi costituita da operazioni di lettura. Inoltre, nella vengono rese note le utilizzazioni medie delle due risorse osservate nello stesso intervallo di tempo: $U_{CPU} = 23\%$ per la CPU e $U_{RAID} = 19\%$ per l'array dischi. Riassumendo, il progetto richiede di costruire un modello di teoria delle code del DBMS, calibrato sulla traccia di workload reale, per ricavare le prestazioni attuali del sistema, il carico massimo sostenibile e l'impatto di un cambio di configurazione dell'array dischi da RAID-0 a RAID-1.

## 2 Numero di core della CPU

Il corretto dimensionamento del numero di core del centro CPU è un passaggio preliminare imprescindibile: il numero di serventi non è solo un parametro architetturale, ma condiziona direttamente come vanno interpretate le utilizzazioni fornite dal progetto e come va calibrata la service demand nelle fasi successive.

### 2.1 Vincolo necessario: capacità elaborativa minima:

Sommando i contributi individuali del tempo di CPU riportato nella traccia per tutte le 100 000 transazioni si ottiene il carico complessivo di lavoro da smaltire:

$$B_{cpu} = \sum_i \text{CPUTime}_i \approx 180\,583 \text{ s} \approx 50.16 \text{ ore} \tag{1}$$

Poiché un singolo core può erogare al più $T = 24$ ore di servizio effettivo in una giornata, e il carico osservato richiede oltre il doppio di tale capacità, è necessario un livello di parallelismo

maggiore. Un primo limite inferiore al numero di core, puramente di fattibilità fisica, è quindi:

$$m_{\min} = \left\lceil \frac{B_{cpu}}{T} \right\rceil = \left\lceil \frac{50.16}{24} \right\rceil = 3 \tag{2}$$

Questo valore, tuttavia, garantisce solo che il sistema sia fisicamente realizzabile. Di conseguenza, non è detto che $m = 3$ sia il numero di core effettivamente installato, né che sia coerente con l'utilizzazione media del 23% dichiarata per il progetto. Determinare $m$ unicamente da questo vincolo, senza un ulteriore confronto con i dati osservati, porterebbe a un dimensionamento privo di una validazione rassicurante.

### 2.2 Determinazione del numero di core tramite calibrazione incrociata:

Il numero di core effettivo può essere ricavato sfruttando due stime indipendenti della service demand $D_{cpu}$ e imponendo che siano coerenti tra loro.

La prima stima è puramente *misurata*: il tempo medio di CPU per transazione riportato nella traccia di workload:

$$\bar{S}_{cpu} = \frac{1}{N} \sum_i \text{CPUTime}_i \approx 1.8058 \text{ s/job}, \tag{3}$$

non dipende in alcun modo dal numero di core, è un dato di servizio grezzo, non filtrato dal modello.

La seconda stima deriva invece dalla *legge dell'utilizzazione per centri multi-servitore*. Il throughput di sistema, calcolato dalla traccia come

$$X_0 = \frac{N}{T} = \frac{100\,000}{86\,400} = 1.157407 \text{ tx/s}, \tag{4}$$

e l'utilizzazione per-core $U_{CPU} = 0.23$ fornita dal progetto sono legati alla service demand dalla relazione

$$U_{CPU} = \frac{X_0 \cdot D_{cpu}}{m} \quad \Longrightarrow \quad D_{cpu}(m) = \frac{U_{CPU} \cdot m}{X_0} \tag{5}$$

che dipende esplicitamente da $m$.

Le due grandezze, $\bar{S}_{cpu}$ e $D_{cpu}(m)$, rappresentano la stessa quantità fisica (la domanda di servizio media della CPU per transazione) stimata con due metodi indipendenti: l'una dalla traccia grezza di workload, l'altra dall'utilizzazione dichiarata. Il valore corretto di $m$ è quello che rende le due stime coerenti:

$$\frac{U_{CPU} \cdot m}{X_0} = \bar{S}_{cpu} \quad \Longrightarrow \quad m = \frac{\bar{S}_{cpu} \cdot X_0}{U_{CPU}} = \frac{1.8058 \times 1.157407}{0.23} \approx 9.087 \tag{6}$$

Arrotondando al numero intero più vicino si ottiene $m = 9$, che rispetta ampiamente il vincolo di fattibilità $m \geq m_{\min} = 3$ individuato al passo precedente. Con $m = 9$ la service demand implicita risulta $D_{cpu}(9) = 0.23 \times 9/1.157407 = 1.7885$ s/job, in accordo (a meno dell'arrotondamento di $m$ a intero) con il valore misurato $\bar{S}_{cpu} = 1.8058$ s/job.

È importante osservare che il criterio basato esclusivamente sul carico complessivo ($m_{\min} = 3$) fornisce una condizione *necessaria ma non sufficiente* per la fattibilità del sistema. Tale valore rappresenta infatti solamente il numero minimo teorico di core richiesto per completare l'elaborazione entro la finestra temporale considerata, ma non descrive la reale configurazione del sistema. L'adozione di soli tre core porterebbe infatti a una service demand della CPU pari a $D_{cpu} = 0.596$ s/job, incompatibile con il tempo medio di CPU osservato nella traccia (1.806 s/job), con una discrepanza di circa un fattore 3. Per tale motivo, nelle analisi successive viene adottato il valore $m = 9$ come numero effettivo di core del centro CPU, in quanto coerente con i dati osservati e con l'utilizzazione dichiarata del sistema.

# 3 Modello del sistema:

Il sistema DBMS viene modellato mediante una rete di code aperta, composta da due centri di servizio disposti in serie, corrispondenti alle due risorse hardware osservate:

- **CPU** – centro multi-servitore costituito da $m = 9$ core paralleli;
- **Array RAID** – centro multi-servitore che rappresenta il sottosistema di memorizzazione, inizialmente configurato come RAID-0 a $n_{disk} = 2$ dischi indipendenti.

Si assume che la stripe unit dell'array sia maggiore della dimensione di pagina del DBMS e che gli accessi siano allineati, cosicché ogni visita logica all'array si traduca in un accesso a un solo disco fisico: è questa ipotesi che rende sensato modellare l'array RAID-0 come una stazione a 2 serventi indipendenti anziché come un unico servente aggregato.

## 3.1 Throughput di sistema:

Il throughput di sistema è già stato ricavato dalla traccia nella Sez. 2.2, in fase di calibrazione del numero di core: $X_0 = N/T = 1.1574$ tx/s. Trattandosi di una misura indipendente dal numero di core, esso costituisce un parametro diretto del modello e viene qui richiamato come base per il calcolo delle service demand nella sezione seguente.

## 3.2 Service demands:

Coerentemente con l'approccio adottato, le service demands non vengono ottenute come semplice media dei tempi riportati nella traccia, ma tramite inversione della legge dell'utilizzazione, usando il throughput e le utilizzazioni medie fornite dal progetto – separando così il dimensionamento hardware (numero di core, numero di dischi) dalla calibrazione del modello prestazionale.

Per il centro CPU, con $m = 9$ servitori paralleli:

$$D_{cpu} = \frac{U_{CPU} \cdot m}{X_0} = \frac{0.23 \times 9}{1.157407} = 1.7885 \text{ s/job} \tag{7}$$

Per il centro RAID, con $n_{disk} = 2$:

$$D_{raid} = \frac{U_{RAID} \cdot n_{disk}}{X_0} = \frac{0.19 \times 2}{1.157407} = 0.3283 \text{ s/job} \tag{8}$$

Dalla service demand del RAID è inoltre possibile ricavare il tempo di servizio del singolo disco, dividendo per il numero medio di visite per transazione osservato in traccia ($\bar{V}_{raid} = 2.7576$ accessi/transazione):

$$S_{raid} = \frac{D_{raid}}{\bar{V}_{raid}} = \frac{0.3283}{2.7576} = 0.1191 \text{ s} \tag{9}$$

Questo valore di $S_{raid}$, calibrato sulla configurazione RAID-0, verrà riutilizzato al Punto 3 per modellare il passaggio a RAID-1, poiché rappresenta una caratteristica fisica del singolo disco e non della particolare organizzazione RAID adottata.

# 4 Tempi di risposta e job nel sistema:

In questa sezione vengono stimati il tempo medio di risposta del sistema e il numero medio di job presenti, due tra i principali indicatori delle prestazioni di un sistema a code. L'analisi viene condotta dapprima mediante un modello analitico semplificato, utilizzato come riferimento iniziale, e successivamente tramite una simulazione realizzata con JMT, in grado di rappresentare correttamente la natura multi-servente della CPU. Il confronto tra i due approcci consente di evidenziare i limiti dell'approssimazione analitica e di individuare una stima più accurata delle prestazioni effettive del sistema.

### 4.1 Stima analitica di riferimento (limite superiore):

Una prima stima, di solo riferimento, si ottiene applicando la formula per il tempo di risposta di un centro a *singolo* servente,

$$R_k = \frac{D_k}{1 - U_k}, \tag{10}$$

sia alla CPU sia al RAID:

$$R_{cpu} = \frac{D_{cpu}}{1 - U_{CPU}} = \frac{1.7885}{1 - 0.23} = 2.3227 \text{ s} \tag{11}$$

$$R_{raid} = \frac{D_{raid}}{1 - U_{RAID}} = \frac{0.3283}{1 - 0.19} = 0.4053 \text{ s} \tag{12}$$

$$R_{TOT} = R_{cpu} + R_{raid} = 2.3227 + 0.4053 = 2.7280 \text{ s} \tag{13}$$

$$N = X_0 \cdot R_{TOT} = 1.157407 \times 2.7280 = 3.157 \text{ job} \tag{14}$$

Questa stima, però, sovrastima sistematicamente il tempo di risposta reale: la formula $R = D/(1 - U)$ presuppone che una richiesta si accodi non appena il servente è occupato, ipotesi corretta per un centro a singolo servente ma non per un centro multi-servitore come la CPU con $m = 9$ core. Con 9 core al 23% di utilizzazione, l'evento "tutti i 9 core occupati contemporaneamente" è raro, per cui la formula a singolo servente è eccessivamente conservativa proprio sul centro dominante del sistema.

### 4.2 Validazione tramite simulazione JMT *(JSIMgraph)*:

Per ottenere una stima corretta si è simulato il modello – rete aperta, Source → CPU (9 serventi) → RAID (2 serventi) → Sink, service demand $D_i$ per stazione – con lo strumento JSIMgraph della suite JMT. I risultati, confrontati con la stima analitica, sono riportati in Tabella 1.

Table 1: Confronto tra stima analitica (singolo servente) e simulazione JMT (multi-servente) – Punto 1

| | Analitico $R = D/(1 - U)$ | JMT (multi-server) | Scarto |
|---|---|---|---|
| $X_0$ [tx/s] | 1.1574 | 1.1392 | 1.6% |
| $R_{TOT}$ [s] | 2.7280 | 2.1235 | 28.5% |
| $N$ [job] | 3.1574 | 2.4242 | 30.2% |
| $U_{CPU}$ | 0.2300 | 0.2245 | 2.4% |
| $U_{RAID}$ | 0.1900 | 0.1891 | 0.5% |

Le utilizzazioni simulate (22.45% CPU, 18.91% RAID) riproducono, entro il rumore di simulazione, quelle misurate dal performance monitor (23%, 19%): tali valori non sono stati imposti ma emergono dalla parametrizzazione, il che valida il modello. Sul tempo di risposta, invece, la formula a singolo servente e la simulazione divergono di circa il 28%, per il motivo discusso sopra.

Il dettaglio per componente conferma la spiegazione (Tabella 2): il tempo di risposta della CPU simulato ($R_{cpu} = 1.7829$ s) è quasi identico alla sua service demand ($D_{cpu} = 1.7885$ s) – la CPU, pur essendo il centro più caricato, non accoda praticamente nulla, perché con 9 core al 23% è estremamente improbabile che tutti i serventi siano occupati contemporaneamente. Il RAID, pur meno utilizzato, ha solo 2 serventi e quindi accumula un'attesa relativa maggiore rispetto alla propria service demand.

La somma dei tempi per componente (1.7829+0.3439=2.1268 s) coincide, entro lo 0.16%, con il tempo di risposta di sistema misurato indipendentemente da JMT (2.1235 s), a conferma della coerenza interna della simulazione.

Table 2: Tempi di risposta per componente – Punto 1

| | Analitico $R = D/(1-U)$ | JMT (multi-server) |
|---|---|---|
| $R_{cpu}$ [s] | 2.3227 | 1.7829 |
| $R_{raid}$ [s] | 0.4053 | 0.3439 |
| Somma [s] | 2.7280 | 2.1268 |

### 4.3 Risposta al Punto 1:

Si assume come risultato del Punto 1 la stima JMT, corretta per la natura multi-servente della CPU: tempo di risposta medio complessivo $R_{TOT} = 2.12$ s ($R_{cpu} = 1.78$ s, $R_{raid} = 0.34$ s), numero medio di job nel sistema $N = 2.42$ (verificato con la legge di Little: $N = X_0 \cdot R = 1.1392 \times 2.1235 = 2.42$).

## 5 Carico massimo per-classe con $U_{bottleneck} \leq 70\%$:

Il secondo punto dell'analisi ha l'obiettivo di determinare il massimo carico di lavoro che il sistema può sostenere mantenendo l'utilizzazione del centro collo di bottiglia non superiore al 70%. A tal fine, il workload viene dapprima suddiviso in classi omogenee mediante tecniche di clustering, così da verificare se le diverse tipologie di transazioni presentino domande di servizio differenti. Successivamente, utilizzando il modello prestazionale ricavato nei capitoli precedenti, viene calcolato il throughput massimo ammissibile e il corrispondente carico per ciascuna classe, verificando inoltre quale centro rimanga il collo di bottiglia del sistema al crescere del carico.

### 5.1 Decomposizione in classi di workload:

Analizzando la traccia con lo strumento di *Workload Analysis* di JMT (clustering k-means a 2 cluster, 50 000 osservazioni ciascuno) emergono due classi di transazioni, di peso identico $p_1 = p_2 = 0.5$, ma con caratteristiche di accesso ai dischi molto diverse in termini di letture/scritture:

Table 3: Classi di workload individuate tramite clustering

| | Classe 1 | Classe 2 |
|---|---|---|
| Peso $p_r$ | 0.50 | 0.50 |
| $V_r$ (visite I/O per transazione) | 2.7552 | 2.7599 |
| %Read$_r$ | 10.02% | 89.96% |
| $D_{cpu,r}$ [s] | 1.7895 | 1.7874 |
| $D_{raid,r}$ [s] | 0.3280 | 0.3286 |

Le due classi hanno cioè domande di servizio praticamente identiche su CPU e RAID (differenza $< 0.2\%$): la classe 1 è a prevalenza di scritture (90%), la classe 2 a prevalenza di letture (90%). Poiché i pesi sono 50/50 e le domande di servizio aggregate coincidono, il modello multi-classe *collassa* su quello mono-classe per tutto ciò che riguarda le utilizzazioni: un incremento proporzionale del carico su tutte le classi non altera il mix, quindi $U_i$ scala dello stesso fattore $\alpha$ indipendentemente dalla decomposizione in classi. La distinzione in classi diventerà invece determinante al Punto 3, dove il rapporto letture/scritture, non l'utilizzazione aggregata, condiziona il costo del mirroring.

Come verifica di coerenza, ricostruendo l'utilizzazione della CPU dal modello multi-classe si ottiene $U_{CPU} = \sum_r D_{cpu,r} \cdot X_{0,r}/m = 0.2300$, identica al valore misurato ($X_{0,r}$ ottenuto come $X_{0,r} = X_0 \cdot p_r$).

### 5.2 Carico massimo sostenibile:

Confrontando le utilizzazioni per-server ($U_{CPU} = 0.23 > U_{RAID} = 0.19$), la CPU resta il collo di bottiglia del sistema. Uno scaling proporzionale del carico mantiene invariato il rapporto tra le utilizzazioni dei due centri, per cui il fattore di scala massimo che porta la CPU al 70% è:

$$\alpha = \frac{U_{\max}}{U_{CPU}} = \frac{0.70}{0.23} = 3.0435 \tag{15}$$

Equivalentemente, invertendo la legge dell'utilizzazione rispetto al throughput:

$$X_{0,\max} = \frac{U_{\max} \cdot m}{D_{cpu}} = \frac{0.70 \times 9}{1.7885} = 3.5225 \text{ tx/s} \quad (\approx 304\,348 \text{ transazioni/giorno}) \tag{16}$$

ripartito equamente tra le due classi: $X_{0,\max,r} = 1.7613$ tx/s ciascuna ($\approx 152\,174$ transazioni/giorno per classe).

A questo livello di carico l'utilizzazione della CPU raggiunge esattamente il 70% (per costruzione), mentre l'array RAID-0 sale a

$$U_{raid}(X_{0,\max}) = \alpha \times U_{RAID} = 3.0435 \times 0.19 = 57.83\%, \tag{17}$$

rimanendo ampiamente sotto soglia: la CPU resta quindi il collo di bottiglia anche al carico massimo, confermando che la scelta di $U_{\max}$ come vincolo sulla CPU era corretta.

## 6 Passaggio a RAID-1:

Il terzo punto dell'analisi valuta l'impatto della sostituzione dell'attuale configurazione RAID-0 con un sistema RAID-1, mantenendo il throughput massimo determinato nel Punto 2 e imponendo che l'utilizzazione del centro disco non superi il limite del 70%. A differenza dei punti precedenti, l'analisi deve tenere conto del diverso comportamento delle operazioni di lettura e scrittura in un array RAID-1: mentre le letture possono essere distribuite tra i dischi del mirror, le scritture devono essere replicate su ogni disco, modificando in modo sostanziale la service demand del centro. Sulla base di tale modello vengono quindi verificati la fattibilità della configurazione, il numero minimo di dischi necessario a rispettare il vincolo prestazionale e la coerenza dei risultati attraverso una simulazione realizzata con JMT.

### 6.1 Modello del centro disco in RAID-1:

Nel passaggio da RAID-0 (striping) a RAID-1 (mirroring a $n$ dischi) il comportamento del centro disco cambia struttura:

- le **letture** possono essere servite da un disco qualunque del mirror: il carico di lettura si distribuisce e si riduce al crescere di $n$;
- le **scritture** devono essere replicate su tutti gli $n$ dischi del mirror: ogni disco sostiene per intero il carico di scrittura, senza alcun beneficio di parallelismo.

Il tempo di servizio del singolo disco $S_{raid} = 0.1191$ s, calibrato al Punto precedente sulla configurazione RAID-0, resta valido: cambia il livello RAID (quante visite fisiche genera una visita logica), non la tecnologia del disco. Senza questa ipotesi il Punto 3 non sarebbe risolvibile, poiché per una configurazione non ancora esistente non si dispone di un'utilizzazione misurata da cui ricavare i demand.

Separando dalla traccia le visite medie di lettura e scrittura per transazione,

$$A = \frac{1}{N}\sum_i \mathrm{IO}_i \cdot \frac{\%\mathrm{Read}_i}{100} = 1.3795 \text{ (letture/tx)} \tag{18}$$

$$B = \frac{1}{N}\sum_i \mathrm{IO}_i \cdot \left(1 - \frac{\%\mathrm{Read}_i}{100}\right) = 1.3781 \text{ (scritture/tx)} \tag{19}$$

con $A + B = 2.7576$, coerente col numero medio di visite osservato. La ripartizione è quindi pressoché paritaria (≈50.03%/49.97%).

L'utilizzazione del centro RAID-1 a $n$ dischi, al throughput $X_0$, risulta:

$$U_{RAID\text{-}1}(n) = \frac{X_0 \cdot S_{raid} \cdot (A + n \cdot B)}{n} = \underbrace{\frac{X_0 \cdot S_{raid} \cdot A}{n}}_{\text{letture}} + \underbrace{X_0 \cdot S_{raid} \cdot B}_{\text{scritture}} \tag{20}$$

Nel termine di scrittura la $n$ si semplifica: il lavoro si moltiplica per $n$ (replica su tutti i dischi) e la capacità per $n$ (numero di serventi), per cui **aggiungere dischi non allevia mai il carico di scrittura**, ma solo quello di lettura. Ne consegue un asintoto invalicabile

$$U_\infty = \lim_{n\to\infty} U_{RAID\text{-}1}(n) = X_0 \cdot S_{raid} \cdot B \tag{21}$$

la cui verifica ($U_\infty \le U_{\max}$) va effettuata *prima* di cercare il valore di $n$, perché se il solo carico di scrittura eccedesse la soglia nessun numero di dischi potrebbe soddisfare il vincolo.

### 6.2 Verifica di fattibilità e dischi necessari:

Valutando il sistema al carico massimo trovato al Punto 2 ($X_{0,\max} = 3.5225$ tx/s):

$$U_\infty = X_{0,\max} \cdot S_{raid} \cdot B = 3.5225 \times 0.1191 \times 1.3781 = 57.80\% \tag{22}$$

Poiché $U_\infty = 57.80\% < 70\% = U_{\max}$, il vincolo di scrittura da solo **non preclude** la fattibilità: esiste un numero finito di dischi che soddisfa il requisito. La feasibility non era tuttavia scontata a priori: se la frazione di scritture del workload avesse superato il valore di breakeven ($U_\infty = U_{\max} \Rightarrow$ frazione di scritture ≈ 60.5% delle visite), nessun $n$ sarebbe stato sufficiente. Il workload analizzato ha il 49.97% di scritture, sotto soglia ma non di molto.

Applicando l'Eq. (20) per $n = 2, \ldots, 9$ dischi si ottiene la Tabella 4.

Table 4: Utilizzazione del centro RAID-1 al variare del numero di dischi del mirror ($X_0 = X_{0,\max}$)

| $n$ dischi | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|
| $U_{RAID\text{-}1}$ | 86.72% | 77.08% | 72.26% | 69.37% | 67.44% | 66.06% | 65.03% | 64.23% |
| Vincolo ≤ 70% | ✗ | ✗ | ✗ | ✓ | ✓ | ✓ | ✓ | ✓ |

Il numero minimo esatto di dischi (a valori reali) si ottiene invertendo l'Eq. (20) rispetto a $n$:

$$n \ge \frac{X_0 \cdot S_{raid} \cdot A}{U_{\max} - X_0 \cdot S_{raid} \cdot B} = \frac{3.5225 \times 0.1191 \times 1.3795}{0.70 - 0.5780} = 4.741 \quad \Longrightarrow \quad n_{\min} = \lceil 4.741 \rceil = 5 \tag{23}$$

in accordo con la Tabella 4: $n = 4$ non basta (72.26%), $n = 5$ soddisfa il vincolo (69.37%).

### 6.3 Risposta al Punto 3:

Sì, è possibile sostenere il throughput $X_{0,\max}$ con RAID-1 mantenendo l'utilizzazione del collo di bottiglia $\leq 70\%$: sono necessari **5 dischi** (mirror a 5 vie). La tolleranza ai guasti richiesta (almeno 1 disco) è ampiamente soddisfatta, poiché un mirror a 5 vie tollera fino a 4 guasti simultanei.

Va tuttavia segnalato che la configurazione è *al limite*: il margine sul vincolo è di soli 0.63 punti percentuali (69.37% contro 70%). Con 6 dischi il margine salirebbe a 2.56 punti percentuali (67.44%) al costo di un solo disco aggiuntivo – una scelta più prudente in un contesto di produzione reale.

Il costo della soluzione va inoltre osservato in termini di capacità utile: servono 5 dischi per un array che, in RAID-0, ne richiedeva 2, con capacità utile pari a quella di un solo disco (fattore di overhead di storage 5×). Il costo del mirroring, inoltre, non è distribuito uniformemente tra le classi di workload: la classe 1 (90% scritture) genera una service demand sul nuovo array pari a $D_{RAID\text{-}1,1} = 1.5087$ s, contro $D_{RAID\text{-}1,2} = 0.4606$ s della classe 2 (90% letture) – un rapporto di 3.28×, invisibile nei Punti 1 e 2 dove le classi erano indistinguibili in termini di utilizzazione aggregata. È proprio la decomposizione in classi effettuata al Punto 2 a rendere possibile attribuire questo costo differenziale (la derivazione di $D_{RAID\text{-}1,r}$ è riportata nel dettaglio in Sez. 7.2).

### 6.4 Validazione tramite simulazione JMT *(JSIMgraph)*:

Il modello RAID-1 a 5 dischi è stato inoltre simulato in JSIMgraph, con tasso di arrivo per classe pari a $X_{0,\max,r} = 1.7613$ tx/s e service demand per classe $D_{RAID\text{-}1,1} = 1.5087$ s, $D_{RAID\text{-}1,2} = 0.4606$ s. Oltre a throughput e utilizzazioni, già confrontabili con il modello analitico come nel Punto 1, JSIMgraph riporta anche gli indici prestazionali globali del sistema – tempo di risposta $R_{TOT}$ e numero medio di job $N$ – direttamente dal report *Global* della simulazione: per un centro multi-servente multi-classe come quello qui analizzato non esiste una forma chiusa altrettanto semplice del $R = D/(1-U)$ usato nel Punto 1, per cui questi due indici sono disponibili solo dalla simulazione e non dal modello analitico. Il confronto completo è riportato in Tabella 5.

Table 5: Confronto tra modello analitico e simulazione JMT – Punto 3 (RAID-1, 5 dischi)

| | Analitico | JMT |
|---|---|---|
| $X_0$ [tx/s] | 3.5225 | 3.5257 |
| $U_{CPU}$ | 0.7000 | 0.7063 |
| $U_{RAID}$ | 0.6937 | 0.7003 |
| $R_{TOT}$ [s] | n.d.* | 3.21 |
| $N$ [job] | n.d.* | 11.26 |

* non disponibile in forma chiusa per un centro multi-servente multi-classe: ricavato unicamente dalla simulazione.

Gli scostamenti tra i due approcci sulle grandezze confrontabili (entro l'1% su $X_0$, $U_{CPU}$, $U_{RAID}$) sono imputabili al rumore di simulazione e non a errori di parametrizzazione, come confermato ricalcolando i service demand impliciti dai risultati JMT tramite la legge dell'utilizzazione: $D_{cpu}$ risulta 1.7736 s (JMT) contro 1.7885 s (modello), $D_{raid}$ risulta 0.3320 s (JMT) contro 0.3283 s (modello).

I valori di $R_{TOT}$ e $N$ riportati in tabella sono a loro volta coerenti tra loro: applicando la legge di Little con il throughput JMT si ottiene $N = X_0 \cdot R_{TOT} = 3.5257 \times 3.21 = 11.31$ job, in accordo (entro l'arrotondamento a due cifre di $R_{TOT}$) con il valore $N = 11.26$ riportato direttamente dal simulatore – la stessa verifica incrociata già effettuata al Punto 1 (Sez. 4.3).

Il passaggio a RAID-1 comporta inoltre un evidente degrado prestazionale rispetto al Punto 1: il tempo di risposta di sistema passa da $R = 2.12$ s (Tabella 1) a $R = 3.21$ s (+51%), e il numero medio di job nel sistema da $N = 2.42$ a $N = 11.26$ – conseguenza diretta dell'aver spinto entrambi i centri vicino alla soglia del 70% di utilizzazione.

### 6.5 Osservazione conclusiva sul RAID-1:

La radice del limite strutturale del RAID-1 è che le scritture si replicano su $n$ dischi e si dividono su $n$ serventi: la $n$ si semplifica e resta un pavimento invalicabile ($U_\infty = 57.80\%$ in questo caso). Una configurazione RAID-10 ($m'$ coppie mirrorate messe in striping tra loro), replicando ogni scrittura su sole 2 unità e distribuendo le coppie via striping, eliminerebbe questo asintoto e ridurrebbe ulteriormente l'utilizzazione per disco a parità di dischi totali. Nel caso qui analizzato il vincolo del progetto impone però esplicitamente RAID-1, e tale configurazione – pur strutturalmente meno efficiente su un workload a scritture significative come questo – risulta comunque sufficiente con 5 dischi, senza necessità di ricorrere al RAID-10.

## 7 Implementazione in JMT:

Il modello è stato realizzato utilizzando due moduli della suite JMT: il modulo di *Workload Analysis* per la decomposizione del carico in classi e il simulatore *JSIMgraph* per tutte le simulazioni (Punti 1 e 3).

### 7.1 Workload Analysis – decomposizione in classi:

Il modulo di Workload Analysis è stato impiegato per la decomposizione del carico nelle due classi già descritta nella Sez. 5.1. Da quell'analisi provengono i parametri per-classe che alimentano il simulatore – i pesi $p_r$, il numero medio di visite $V_r$ e la frazione di letture $\%\text{Read}_r$ – dai quali sono state ricavate analiticamente le service demand per classe.

### 7.2 JSIMgraph – Punto 1 e Punto 3:

Tutte le simulazioni adottano la stessa topologia, minimale: *Source* → Queue "CPU" → Queue "RAID" → *Sink*, con due classi aperte. Ogni stazione ha una sola visita e tempo di servizio esponenziale pari direttamente alla service demand per classe $D_{i,r}$: ciò è lecito perché $D_i = S_i V_i$ aggrega già tutte le visite nel demand, che diventa così l'input diretto del modello (coerentemente con l'Esempio 8 delle slide).

- **Punto 1** (RAID-0): CPU con *Number of Servers*= 9, RAID con *Number of Servers*= 2; arrival rate per classe $\lambda_r = X_0/2 = 0.5787$ tx/s; service time $D_{cpu,r} = 1.7895$ / $1.7874$ s sulla CPU e $D_{raid,r} = 0.3280$ / $0.3286$ s sul RAID. La simulazione restituisce $U_{CPU} \approx 0.22$ e $U_{RAID} \approx 0.19$: le utilizzazioni misurate vengono riprodotte, non imposte, ed è questa la validazione del modello.
- **Punto 3** (RAID-1, 5 dischi): stessa struttura, con la sola Queue "RAID" riconfigurata a *Number of Servers*= 5; arrival rate per classe $\lambda_r = X_{0,\max}/2 = 1.7613$ tx/s; service time sul RAID pari alle service demand per classe $D_{RAID\text{-}1,1} = 1.5087$ s e $D_{RAID\text{-}1,2} = 0.4606$ s. La CPU resta invariata.

Le due service demand del Punto 3 si ottengono generalizzando alle singole classi lo stesso principio già usato per ricavare l'Eq. (20): separare il contributo delle letture, che sfruttano il parallelismo del mirror, da quello delle scritture, che invece lo annullano perché devono essere replicate su ogni disco.

Per la classe $r$, la Workload Analysis fornisce (Tabella 3) il numero medio di visite I/O per transazione $V_r$ e la frazione di letture $\rho_r = \%\text{Read}_r/100$. Una transazione della classe $r$ genera quindi, in media, $V_r\rho_r$ visite di lettura e $V_r(1-\rho_r)$ visite di scrittura. Come nel caso aggregato, ciascuna visita di lettura produce una sola visita fisica (un disco qualsiasi del mirror la può servire), mentre ciascuna visita di scrittura, dovendo essere replicata su tutti gli $n$ dischi, ne produce $n$. Il lavoro fisico totale generato sull'intero array da una transazione di classe $r$ è pertanto

$$D_{RAID\text{-}1,r} = S_{raid} \cdot \Big[ \underbrace{V_r\rho_r}_{\text{letture}} + \underbrace{n \cdot V_r(1-\rho_r)}_{\text{scritture}} \Big] = S_{raid} \cdot V_r \cdot \big[\rho_r + n\,(1-\rho_r)\big], \tag{24}$$

espressione che, a differenza dell'Eq. (20), non viene divisa per $n$: è infatti la service demand *totale* della classe da fornire al simulatore, che applicherà internamente la divisione per $n$ tramite la stessa legge dell'utilizzazione multi-servente $U = X_0 D/n$ usata in tutto il resto del report. Con $S_{raid} = 0.1191$ s ed $n = 5$ dischi si ottiene:

$$D_{RAID\text{-}1,1} = 0.1191 \times 2.7552 \times [0.1002 + 5(0.8998)] = 1.5087 \text{ s (classe write-heavy)},$$

$$D_{RAID\text{-}1,2} = 0.1191 \times 2.7599 \times [0.8996 + 5(0.1004)] = 0.4606 \text{ s (classe read-heavy)}.$$

Il termine $[\rho_r + n(1-\rho_r)]$ agisce come un fattore di amplificazione dovuto al mirroring: vale 1 per una classe di sole letture ($\rho_r = 1$, nessuna penalità) e sale fino a $n$ per una classe di sole scritture ($\rho_r = 0$, penalità massima). La classe 1, quasi tutta scritture ($\rho_1 = 10.02\%$), si trova vicina a quest'ultimo estremo, mentre la classe 2, quasi tutta letture ($\rho_2 = 89.96\%$), è vicina al primo: è questa asimmetria, già presente nei dati ma invisibile a livello di utilizzazione aggregata, a produrre il rapporto 3.28× tra le due service demand osservato in Sez. 6.3. Fornendo al simulatore direttamente i $D_{RAID\text{-}1,r}$ così calcolati, la stazione multi-servitore a 5 dischi riproduce correttamente l'utilizzazione del centro senza necessità di una topologia dedicata per letture e scritture.

# 8 Conclusioni:

L'analisi svolta ha permesso di caratterizzare il comportamento prestazionale del sistema e di valutarne la capacità di sostenere incrementi di carico e modifiche architetturali. In primo luogo, la sola somma dei tempi di CPU presenti nella traccia, pari a circa 50.16 ore, dimostra che almeno tre core rappresentano un requisito necessario affinché l'elaborazione possa essere completata entro una finestra temporale di 24 ore. Tuttavia, imponendo la coerenza tra il tempo medio di CPU osservato nella traccia (1.806 s per transazione) e la service demand ricavata dall'utilizzazione dichiarata del 23%, si ottiene una stima più realistica della configurazione del sistema, pari a nove core.

A partire dal throughput di sistema, pari a $X_0 = 1.1574$ transazioni al secondo, le service demand dei due centri sono stati determinati applicando la legge dell'utilizzazione, ottenendo $D_{cpu} = 1.7885$ s e $D_{raid} = 0.3283$ s. Tale approccio consente di ricavare la reale service demand, evitando di basarsi esclusivamente sui tempi medi grezzi riportati nella traccia CSV.

La modellazione del sistema tramite una rete di code, simulata mediante JMT, evidenzia un tempo medio di risposta pari a 2.12 s e un numero medio di job nel sistema di circa 2.42. Il confronto con il modello analitico a singolo servente mostra come quest'ultimo sovrastimi entrambe le grandezze di circa il 28–30%, confermando che tale approssimazione non è adeguata per descrivere un centro multi-servitore poco carico come la CPU a nove core. Nonostante ciò, la CPU rimane il collo di bottiglia del sistema, con un'utilizzazione del 23% per core, superiore a quella del RAID-0, pari al 19

L'analisi di scalabilità mostra inoltre che il sistema può sostenere un incremento uniforme del carico fino a circa 3.04 volte quello attuale, raggiungendo un throughput massimo di circa

3.52 transazioni al secondo, corrispondente a circa 304,348 transazioni al giorno, senza superare il limite del 70% di utilizzazione per core sulla CPU. In tali condizioni, il RAID-0 mantiene comunque un'utilizzazione inferiore alla soglia fissata, attestandosi al 57.83

Infine, è stata valutata la sostituzione del RAID-0 con una configurazione RAID-1. L'analisi dimostra che questa soluzione risulta sostenibile dal punto di vista prestazionale soltanto adottando un mirror composto da cinque dischi: una configurazione a due dischi porterebbe infatti l'utilizzazione fino all'86.7%, superando il limite imposto. Con cinque dischi, invece, il sistema rispetta il vincolo del 70% e garantisce la tolleranza al guasto di almeno un disco, arrivando a sopportarne fino a quattro. Tale soluzione comporta un costo di storage pari a cinque volte quello di un singolo disco e lascia un margine operativo piuttosto ridotto (0.63 punti percentuali), ma rende superfluo il ricorso a configurazioni RAID-10, inizialmente considerate in una valutazione più conservativa.